

Analysis Report

Student Full Name	Sinhalage Viraj Deshappriya Jayasiri	Student ID No.	w2153009 / 20241154
-------------------	--------------------------------------	----------------	---------------------

Case Study Title

Case Study (A): Predicting Clients Loan Status.

[Total 43 Marks]

Research Question: Does machine learning have the potential to assist banker and finance analysts in predicting which client can be approved a loan?

Task (1) – Domain Understanding: Classification

The financial risk analysts decided that classification modelling is required. Indicate in the table below for each of the listed variables in your data which ones you should RETAIN and can be included in the classification modelling of loan approval status, and the variables you should DROP (REMOVE). Justify your decision logically and/or by research (for any research, include in-text citation using the Harvard style).

[3 Marks]

Variable Name	RETAIN or DROP	Brief justification for retention or dropping with in-text citation.
ID	DROP	ID is only a unique identifier and does not describe borrower risk. It has No predictive meaning for loan approval and may introduce noise if used in modelling (James et al., 2021).
Sex	DROP	This variable is not present in the supplied dataset used in the notebook. A variable that is unavailable in the provided data cannot be included in modelling.
Age	RETAIN	Age may influence financial stability, credit behaviour, and lending risk, so it is a relevant predictor for approval decisions.
Education Qualifications	DROP	This variable is not present in the supplied dataset used in the notebook, so it cannot be included in the final classification model.
Income	RETAIN	Income is directly linked to repayment capacity and affordability, so it is a core lending feature.
Home Ownership	RETAIN	Housing status provides information about financial stability and may Reflect borrowing risk.
Employment Length	RETAIN	Employment history is a useful indicator of income stability and Repayment reliability.
Loan Intent	RETAIN	The reason for borrowing can affect risk, since different loan purposes May be associated with different repayment behaviour.
Loan Amount	RETAIN	Requested loan size is directly relevant to approval decisions because higher borrowing may increase risk.
Loan Interest Rate	RETAIN	Interest rate reflects lending conditions and risk profile and is useful for approval modelling.
Loan-to-Income Ratio (LTI	RETAIN	LTI captures affordability by relating the requested loan to income and is highly relevant to approval decisions.

Payment Default on File	RETAIN	A previous default is a strong indicator of borrower credit risk and is Therefore highly relevant.
Credit History Length	RETAIN	Credit history length provides evidence about past borrowing experience lender confidence.
Loan Approval Status	RETAIN	This is the classification target variable that the model must predict.
Maximum Loan Amount	DROP	Maximum Loan Amount is the regression target for Case Study B. Keeping it in the classification model would create target leakage.
Credit Application Acceptance	DROP	This variable is not present in the supplied notebook dataset, and it also overlaps conceptually with approval status.

References

James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning*. 2nd edn. New York: Springer.

Kuhn, M. and Johnson, K. (2019) *Feature Engineering and Selection*. Boca Raton: CRC Press.

Task (2) – Exploring and Understanding Your Dataset

With the aid of your Final Python Notebook 1, for your RETAINED input variables and your class “Target” variable, produce 1)**basic descriptive stats** and 2)**variable scale type**, then 3)**plot the distribution of your target variable**. (Paste the three screenshots of code OUTPUTS ONLY for evidence of these elements).

[2 Marks]

Basic Descriptive Statics

	id	age	income	employment_length	loan_amount	loan_interest_rate	loan_income_ratio	credit_history_length	loan_approval_status	max_allowed_loan
count	58645.000000	58639.000000	5.864500e+04	58645.000000	58645.000000	58634.000000	58645.000000	58645.000000	58645.000000	5.864500e+04
mean	29322.000000	27.550913	6.404617e+04	4.703487	9217.556518	10.677526	0.159238	5.813556	0.142382	6.975472e+04
std	16929.497605	6.033217	3.793111e+04	4.004982	5563.807384	3.036034	0.091692	4.029196	0.349445	6.175091e+04
min	0.000000	20.000000	4.200000e+03	0.000000	500.000000	-11.140000	0.000000	2.000000	0.000000	-2.426900e+06
25%	14661.000000	23.000000	4.200000e+04	2.000000	5000.000000	7.880000	0.090000	3.000000	0.000000	3.800300e+04
50%	29322.000000	26.000000	5.800000e+04	4.000000	8000.000000	10.750000	0.140000	4.000000	0.000000	6.239200e+04
75%	43983.000000	30.000000	7.560000e+04	7.000000	12000.000000	12.990000	0.210000	8.000000	0.000000	9.271600e+04
max	58644.000000	123.000000	1.900000e+06	150.000000	35000.000000	23.220000	0.830000	30.000000	1.000000	2.638778e+06

Figure 1: Basic descriptive statics

Identify Variable Type

```
Data Types:
id                int64
age               float64
income            int64
home_ownership    object
employment_length int64
loan_intent        object
loan_amount        int64
loan_interest_rate float64
loan_income_ratio  float64
payment_default_on_file object
credit_history_length int64
loan_approval_status int64
max_allowed_loan  int64
dtype: object
```

Figure 2: Identify variable type

Plot Distribution of Target Variables

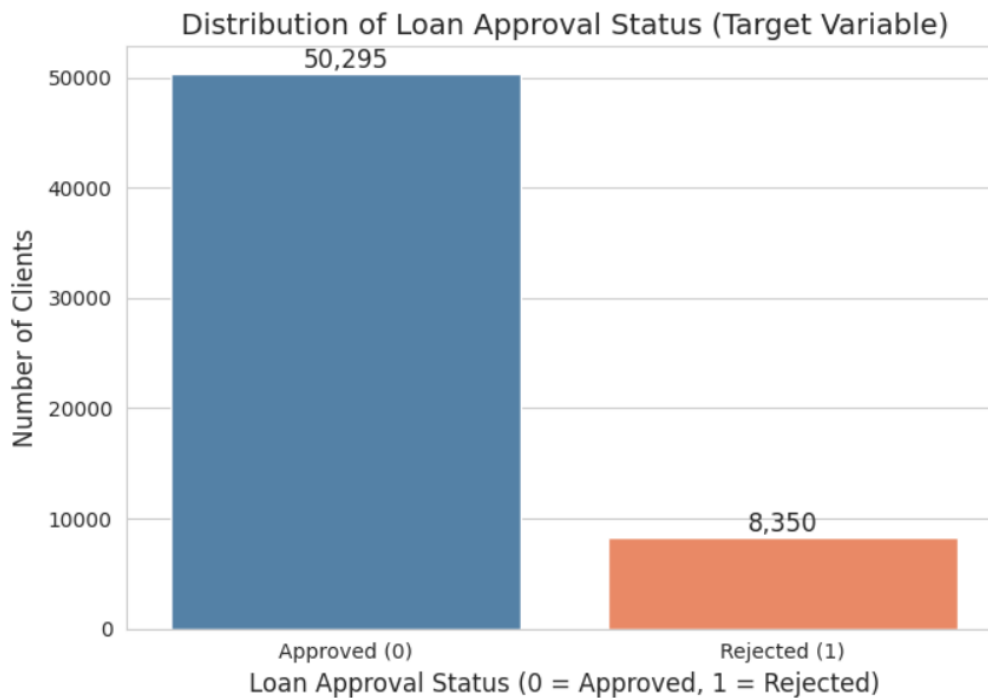


Figure 3: Plot distribution of target variables

Task

(3) -

Data Preparation: Cleaning and Transforming your data

a) With the aid of your Final Python Notebook 1, when you first explored your retained variables in the loan dataset, you may have found some issues. **1) Report any issues you found in your retained dataset variables.** Based on the issues you found in your data, **2)suggest a suitable possible method to fix each of these issues** and **3)provide your justification for using your suggested fix method.** Use the table below to organise your findings and analysis, and add more rows if needed

[4 Marks]

	Variable Name	Issue found	Proposed fix	Justification for used fix method
1	age	One age value above 100 years was detected, which is biologically implausible in this consumer lending dataset.	Replace values above 100 with NaN, then impute using the median.	Median imputation is robust to skew and avoids allowing an unrealistic outlier to distort the distribution.
2	loan_interest_rate	Missing values were present.	Impute missing values using the median.	The median preserves the central tendency of numeric data without being overly influenced by extreme values.
3	payment_default_on_file	Missing values were present in a categorical variable.	Impute using the mode.	Mode imputation is appropriate for categorical data because it preserves the most common valid category.
4	max_allowed_loan	Negative values were detected, which are invalid for a maximum loan amount.	Remove rows with negative values.	A negative maximum lending value is a data entry error rather than a meaningful financial observation.
5	home_ownership / loan_intent	Categorical values cannot be used directly by the modelling algorithms in later notebooks.	Encode these variables numerically using LabelEncoder.	The notebook's chosen modelling workflow requires numeric inputs for classification and regression algorithms.

Task (3) – Data Preparation: Cleaning and Transforming your data

b) With the aid of Python packages and your Final Python Notebook 1, implement your suggested fixes of issues in the previous Task 3-a in your final Python Notebook1.

1) Show evidence (before and after) of implementing your suggested fix to the problems you identified for your dataset in Task 3-a.

To show your evidence, paste screenshots of your relevant code OUTPUTS ONLY (Do not paste the code).

2) Indicate and annotate the issue and the fix in each of your provided evidence screenshots.

[4 Marks]

Output screenshot of the issue before the fix	Output screenshot after fixing the issue
---	--



Figure 4: Before fixing age boxplot showing the implausible outlier

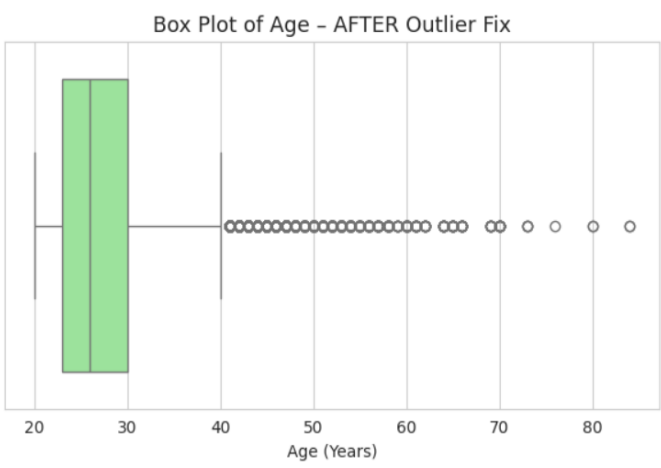


Figure 5:After fixing age outlier removed and age imputed

Output screenshot of the issue before the fix	Output screenshot after fixing the issue
---	--

```
Missing values AFTER ALL fixes:
id                                0
age                               0
income                           0
home_ownership                   0
employment_length                0
loan_intent                      0
loan_amount                      0
loan_interest_rate              0
loan_income_ratio               0
payment_default_on_file         0
credit_history_length           0
loan_approval_status            0
max_allowed_loan                0
dtype: int64

Total missing values: 0
```

Figure 7:After fixing missing values

```
Missing values AFTER ALL fixes:
id                                0
age                               0
income                           0
home_ownership                   0
employment_length                0
loan_intent                      0
loan_amount                      0
loan_interest_rate              0
loan_income_ratio               0
payment_default_on_file         0
credit_history_length           0
loan_approval_status            0
max_allowed_loan                0
dtype: int64

Total missing values: 0
```

Figure 6: Figure 5:Before fixing missing Value

Task (4) – Classification Modelling of Clients Loan Approval Status

a) In your Final Python Notebook 2, you built THREE different models to predict loan approval status: Logistic Regression (LR), K Nearest Neighbour (KNN) and Naïve Bayes (NB). These algorithms are a mix of parametric and non-parametric algorithms.

1) **Note down the type of each algorithm (parametric vs non-parametric),**

2) **name any learnable parameters,** and

3) **list any strategic hyperparameters for each algorithm** which you want to consider tuning. Organise your answer in the table below:

[3 Marks]

Algorithm Name	Algorithm Type	Learnable Parameters	Some Strategic Hyperparameters
Naïve Bayes (NB)	Linear Classifier	Class prior probabilities, mean and variance per class/feature in Gaussian NB	var_smoothing
Logistic Regression (LR)	Probabilistic Classifier	Regression coefficients/weights and intercept	C, penalty, solver, max_iter
K-Nearest Neighbour (KNN)	Instance-based Learner	No explicit learned coefficients; stores training instances and uses distance-based voting	n_neighbors, metric, weights

Task (4) – Classification Modelling of Clients Loan Approval Status

b) With the aid of your Final Python Notebook 2, use the training–test split approach with your retained applicable input features only and the target output feature to build your predictive classification models.

[3 Marks]

i. Screenshot

1) **the list of all feature names used for building your classification models** and the corresponding

2) **data shape function output.** (Paste screenshots of the relevant code output only; do not paste the Python code).

```
Feature Names used for Classification Modelling:
['age', 'income', 'home_ownership_enc', 'employment_length', 'loan_intent_enc', 'loan_amount', 'loan_interest_rate', 'loan_income_ratio', 'payment_default_on_file', 'credit_

Data Shape (X): (58642, 10)
Target Shape (y): (58642,)

Target class distribution (0=Approved, 1=Rejected):
loan_approval_status
0    50292
1     8350
Name: count, dtype: int64
```

Figure 8:Output evidence: classification feature list and data shape

ii. In less than 150 words, **research and justify (defend) your choice of the training-test split ratio** and provide an in-text citation.

An 80:20 training–test split was used. It is a normal and justifiable decision since it is giving the model a big enough training set to learn useful patterns, but at the same time retains a big enough independent test set to use to measure the performance of the model. An applied machine learning test set needs to be sufficiently large to provide a consistent estimate of the out of sample performance, but not so large that it undermines the training process. In the case of a relatively large dataset like this one, an 80:20 split provides a good balance between learning the model and the reliability of its evaluation (James et al., 2021).

References

James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning*. 2nd edn. New York: Springer.

iii. Provide as evidence **the code block line and code output from your Final Python Notebook 2** that ensures two conditions:

- 1) **all your models were tested on the same test instances (clients) in your dataset;**
- 2) **the labels ratio of approval Status “Accept” to “Reject” is the same in the training and test subsets.**
- 3) **State the training-test split function parameters in your code line that are responsible for meeting both conditions.**
- 4) **In less than 150 words, research and justify (defend) your decision to implement both conditions in your Python Notebook 2 notebook with in-text citations where possible.**

```
# Split the scaled data into training (80%) and test (20%) sets
# random_state=42 ensures all models share the same test instances (reproducibility)
# stratify=y ensures the class label ratio is preserved in both train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Print the sizes of the resulting subsets
print('Training set shape:', X_train.shape)
print('Test set shape: ', X_test.shape)

# Print the class distribution in training and test sets to confirm stratification
print('\nTraining target distribution (0=Approved, 1=Rejected):')
print(pd.Series(y_train).value_counts())
print('\nTest target distribution (0=Approved, 1=Rejected):')
print(pd.Series(y_test).value_counts())
```

Figure 9: Code evidences-Train_test_split parameters used in Notebook 2

```
Training set shape: (46913, 10)
Test set shape: (11729, 10)

Training target distribution (0=Approved, 1=Rejected):
loan_approval_status
0    40233
1     6680
Name: count, dtype: int64

Test target distribution (0=Approved, 1=Rejected):
loan_approval_status
0    10059
1     1670
Name: count, dtype: int64
```

Figure 10: Output evidence-same test instances and preserved class ratios in train/test splits

This line guarantees two important stipulations. First, `random_state=42` makes sure that the splits are replicated each time, that is, all three models are run on the same test clients. Second, `stratify=y` maintains the proportion of Approved (0) and Rejected (1) cases in the training and test set. These two controls enhance fairness of comparison and enhance reliability of the evaluation especially in cases where the distribution of the classes is imbalanced. The models require consistent test cases to compare them equally, and stratification eliminates biasing class ratios that may give false performance measures (Kuhn and Johnson, 2019).

References with in-text citation

Kuhn, M. and Johnson, K. (2019) *Feature Engineering and Selection*. Boca Raton: CRC Press.

Task (5) – Evaluating your Loan Approval Status Classification Models

Your finance team provided the following success criteria to guide you when evaluating and selecting your best model: ***“When evaluating your model’s performance, which addresses your first research question (a). The model is expected to misclassify clients’ applications. Thus, the model should aim to predict as many “Reject” status for clients as many as possible to decrease the risk of future defaulted loan payments. However, the model should demonstrate that its high “Reject” prediction rate is mainly due to a larger number of correctly detected (predicted) rejected loan applications.”***

a) With the aid of Final Python Notebook 2, for each of your models (Logistic Regression LR, Naive Bayes NB and K-Nearest Neighbours KNN)

- 1) **paste the test confusion matrix,**
- 2) **the classification report and**
- 3) **the AUC-ROC curve graphs**

as screenshots from the output of your Python code.

[3 Marks]

1) Screenshot of the test confusion matrix for (NB, LR, and KNN); make sure you title each matrix with its algorithm name

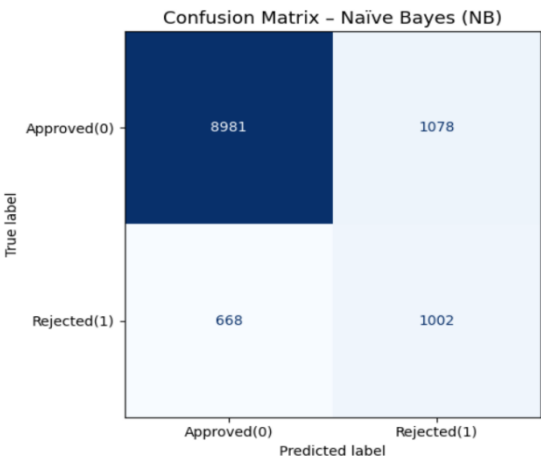


Figure 11: NB confusion matrix

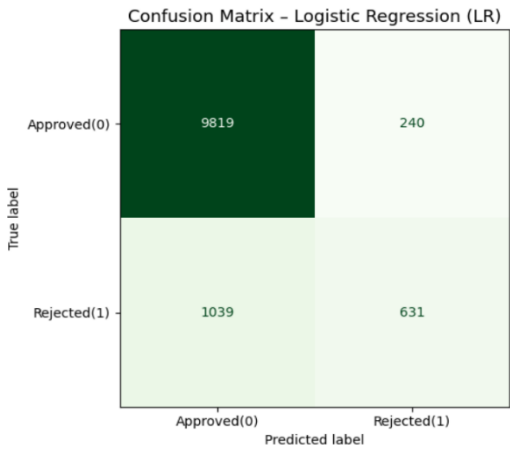


Figure 12:LR confusion matrix

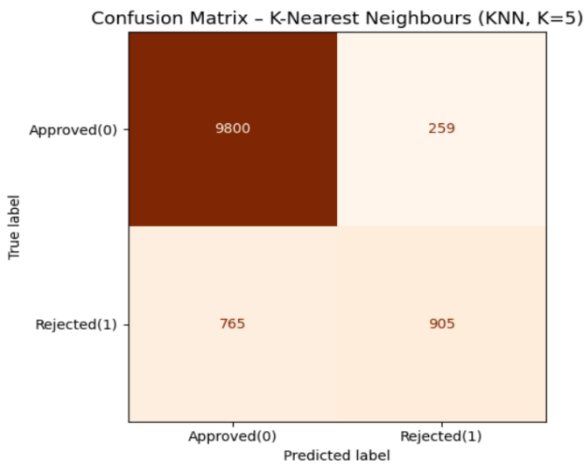


Figure 13: KNN confusion matrix

2) Screenshot of the classification report for (NB, LR, and KNN); make sure you title each classification report with its algorithm name

Classification Report - Naïve Bayes (NB):

	precision	recall	f1-score	support
Approved(0)	0.93	0.89	0.91	10059
Rejected(1)	0.48	0.60	0.53	1670
accuracy			0.85	11729
macro avg	0.71	0.75	0.72	11729
weighted avg	0.87	0.85	0.86	11729

Accuracy: 0.8511
Precision: 0.4817
Recall: 0.6000
F1-Score: 0.5344
AUC-ROC: 0.8558

Figure 14:NB classification report

Classification Report - Logistic Regression (LR):

	precision	recall	f1-score	support
Approved(0)	0.90	0.98	0.94	10059
Rejected(1)	0.72	0.38	0.50	1670
accuracy			0.89	11729
macro avg	0.81	0.68	0.72	11729
weighted avg	0.88	0.89	0.88	11729

Accuracy: 0.8910
Precision: 0.7245
Recall: 0.3778
F1-Score: 0.4967
AUC-ROC: 0.8730

Figure 15: LR classification report

Classification Report - K-Nearest Neighbours (KNN, K=5):

	precision	recall	f1-score	support
Approved(0)	0.93	0.97	0.95	10059
Rejected(1)	0.78	0.54	0.64	1670
accuracy			0.91	11729
macro avg	0.85	0.76	0.79	11729
weighted avg	0.91	0.91	0.91	11729

Accuracy: 0.9127
Precision: 0.7775
Recall: 0.5419
F1-Score: 0.6387
AUC-ROC: 0.8545

Figure 16:KNN classification report

3) Screenshot of the AUC-ROC Curve for (NB, LR, and KNN); make sure you title each graph with its algorithm name

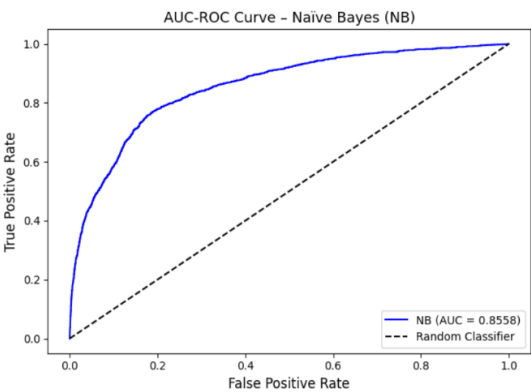


Figure 17:NB AUC-ROC curve

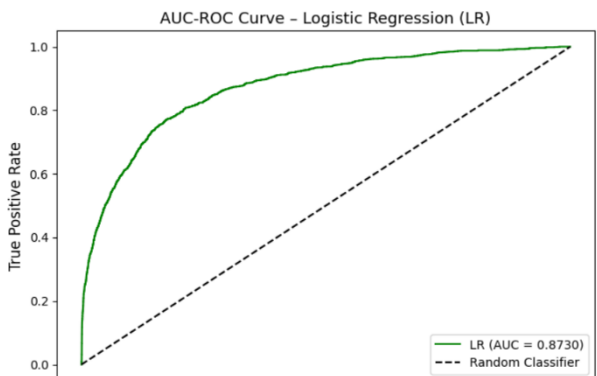


Figure 18:LR AUC-ROC curve

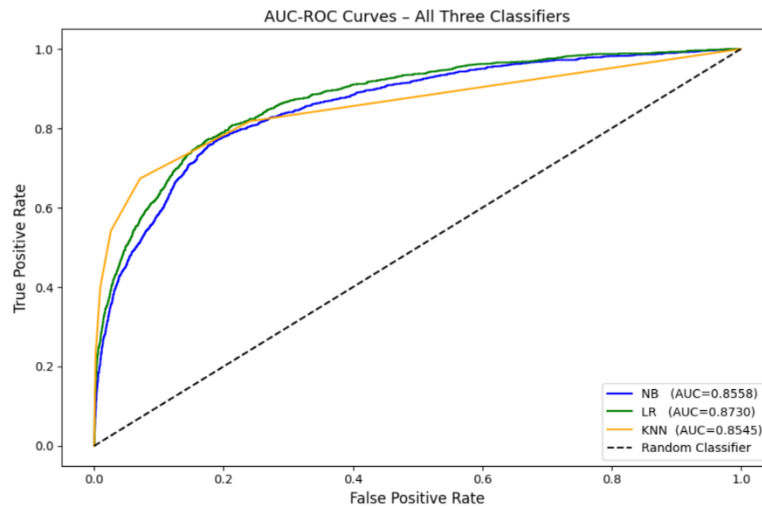


Figure 19: KNN AUC-ROC curve

Task (5) – Evaluating your Loan Approval Status Classification Models

b) Five different classification evaluation metrics are calculated in your Final Python Notebook 2.

1) State which evaluation metric/metrics to “USE or “DO NOT USE” to closely interpret the above success criteria.

2) For justification, explain how closely your choice of “USE” or “DO NOT USE” for a metric interprets the given success criteria.

With the aid of your Final Python Notebook 2,

3) document all the **TEST SCORES** for each built model in the table below.

[7 Marks]

Metric	1) USE or DO NOT USE	2) Justification for choosing “USING” or “NOT USING” this metric in relation to the success criteria	Model	3) Metric Test Score
Accuracy	DO NOT USE	Here, accuracy can be misleading because the dataset has more approved loans than rejected loans. A model could be very accurate and still not do a good job of finding applicants who were turned down.	NB	0.8511
			LR	0.8910
			KNN (k=5)	0.9127
Recall	USE	The finance team wants the model to find as many rejected applications as possible, so recall for the rejected class is important.	NB	0.6000
			LR	0.3778
			KNN (k=5)	0.5419
Precision	USE	It is also important that precision is taken into account since the brief states that the high reject prediction rate should primarily be due to the accurate detected rejected applications instead of false alarms.	NB	0.4817
			LR	0.7245
			KNN (k=5)	0.7775
F1-score	USE	F1-score is a balance between recall and precision, and is appropriate when it is important to both capture rejected clients and not to get too many false positives.	NB	0.5344
			LR	0.4967
			KNN (k=5)	0.6387
AUC-Roc	USE	AUC-ROC assesses discrimination between classes at thresholds and can be used to evaluate how the model can discriminate between approved and rejected clients.	NB	0.8558
			LR	0.8730

			KNN (k=5)	0.8545
--	--	--	-----------	--------

Task (5) – Evaluating your Loan Approval Status Classification Models

c) **Suggest a single best approval status classification model** based on the 'USED' performance metrics scores you identified in Task (5-b). In less than 100 words, briefly **describe how well your best model satisfies the needs of your finance team**.

[2 Marks]

Naive Bayes was chosen as the best baseline model, based on the main focus on recall of rejected applications, as it presented the best recall of the rejected-class (0.6000). This implies that it identified the highest proportion of rejected applicants, which is in line with the objectives of the finance team to minimize risk of default in the future. Even though its accuracy was worse than KNN and LR, it was the most suitable to the mentioned priority of maximising rejected-case detection prior to tuning.

Task (5) – Evaluating your Loan Approval Status Classification Models

d) To enhance your selected best model/s performance (from Task 5-c), tune some of its possible hyperparameters, which you indicated in Task (4-a) for that specific algorithm. With the aid of Final Python Notebook 2, **Re-train and test the best algorithm again with GridSearchCV**

[5 Marks]

i. With the aid of your Final Python Notebook 2,

1) Paste into this report the line of code which shows evidence of specifying a parameters grid and applying the GridSearchCV function to rebuild your selected best model.

2) Then, document the estimated best hyperparameters for the optimised model.

```
# Define the hyperparameter grid for Naïve Bayes
# var_smoothing controls the portion of the largest variance added to all variances for stability
param_grid_nb = {
    'var_smoothing': np.logspace(-12, 0, 13)
}

# Instantiate GridSearchCV to search for the best var_smoothing value
# cv=5: 5-fold cross-validation on the training set
# scoring='recall': optimise for Recall (success criteria metric for rejected class)
gs_nb = GridSearchCV(
    GaussianNB(),
    param_grid_nb,
    cv=5,
    scoring='recall',
    n_jobs=-1
)

# Fit GridSearchCV on the training data to find optimal hyperparameters
gs_nb.fit(X_train, y_train)

# Print the best hyperparameter values found by GridSearchCV
print('GridSearchCV - Best Hyperparameters for Naïve Bayes:')
print(gs_nb.best_params_)
print(f'Best Cross-Validated Recall Score: {gs_nb.best_score_:.4f}')
```

Figure 20: Code evidence- parameter grid and GridSearchCV for Naive Bayes

```
GridSearchCV - Best Hyperparameters for Naïve Bayes:
{'var_smoothing': np.float64(1e-12)}
Best Cross-Validated Recall Score: 0.6162
```

Figure 21: Output evidence- best hyperparameters found for Naive Bayes

ii. With the aid of your Final Python Notebook 2,

1) paste into this report the test confusion matrix for your best model before and after hyperparameter tuning.

2) Also, document the new score/s of the "USED" performance metric/s of your choice to interpret the success criteria indicated in Task (5.b) before and after tuning.

3) Comment on whether the tuning of hyperparameters of your best model improved its positive predictive ability in line with the success criteria.

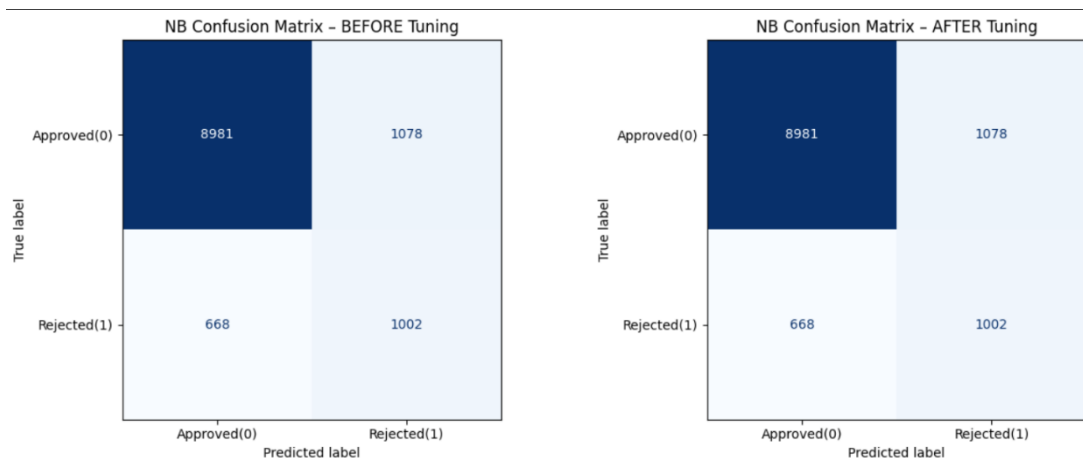


Figure 22: Confusion matrices before and after tuning

Metrics Comparison - Naive Bayes Before vs After GridSearchCV Tuning:		
Metric	Before	After
Recall (Rejected)	0.6000	0.6000
Precision	0.4817	0.4817
Accuracy	0.8511	0.8511
AUC-ROC	0.8558	0.8558

Figure 23 : Metric comparison before and after tuning

Comment:

Compare whether tuning improved rejected-class recall and whether any gain in recall came at the cost of a large fall in precision. If recall improved without an unacceptable decline in precision, the tuning can be considered beneficial for the success criterion.

Task (5) – Evaluating your Loan Approval Status Classification Models

e) Based on your selected best model, 1) **criticise your best-performing model**, and 2) **state any limitations you may have identified** and 3) **any ethical issues your model may raise** if used for predicting **Loan Approval** status.

[2 Marks]

There are various limitations to the selected model. Firstly, Naive Bayes makes the assumption of conditional independence of predictors, which is not likely to be the case in real lending data where predictors include income, loan amount, and loan-to-income ratio are connected. Second, the model relies on coded categorical variables, which can simplify intricate borrower behaviour. Third, the model is less precise, and it could therefore falsely categorize the number of clients who are approved as rejected. Morally, an approval model can enhance bias when historical lending decisions have been also biased. This may disavowable discriminate some groups in the event that the model is applied without transparency, monitoring and checks of fairness.

Task (5) – Evaluating your Loan Approval Status Classification Models

f) With the aid of your Final Python Notebooks 3, combine only TWO out of the THREE base learners (NB, LR, KNN) that you already built into a probability-based voting ensemble classifier.

[5 Marks]

i. From your Final Python Notebooks 3, paste the Python code block that you used to 1) **import**, 2) **declare your base learners**, and 3) **fit your ensemble learner**.

```

# Import VotingClassifier for ensemble learning
from sklearn.ensemble import VotingClassifier

# Declare the two base learners for the ensemble
base_learner_nb = GaussianNB()
base_learner_lr = LogisticRegression(max_iter=1000, random_state=42)

# Instantiate the probability-based soft voting ensemble classifier
# voting='soft' uses predicted class probabilities (averages them) for final prediction
ensemble = VotingClassifier(
    estimators=[
        ('nb', base_learner_nb),
        ('lr', base_learner_lr)
    ],
    voting='soft'
)

# Fit (train) the ensemble classifier on the training set
ensemble.fit(X_train, y_train)

print('Soft Voting Ensemble (NB + LR) trained successfully.')

```

Figure 24: Code evidence- imported, declared, and fitted soft-voting ensemble

ii. In this analysis report,

1) paste the test confusion matrices, 2) AUC-ROC Curves and 3) the classification reports for each of the TWO base learners you chose to combine,
as well as **4) the test confusion matrix, 5) classification report for the voting Ensemble Learner and 6) the AUC-ROC curve for the ensemble learner.**

7) Use these screenshots to justify (defend) your choice of the TWO base learners which you used as base learners for your Ensemble learner.

1) Paste the test confusion matrices for each base learner (2 x matrices)

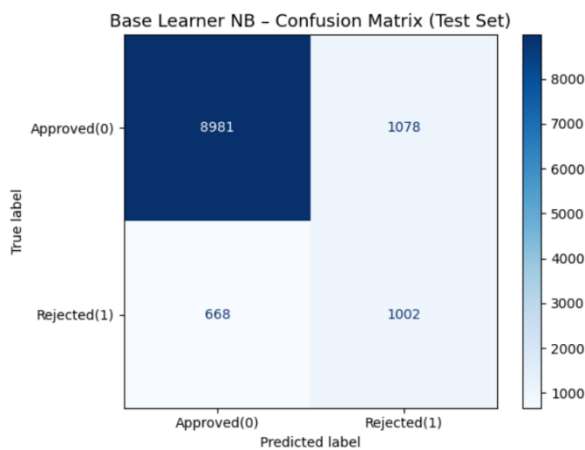


Figure 25: Confusion Matrix - NB

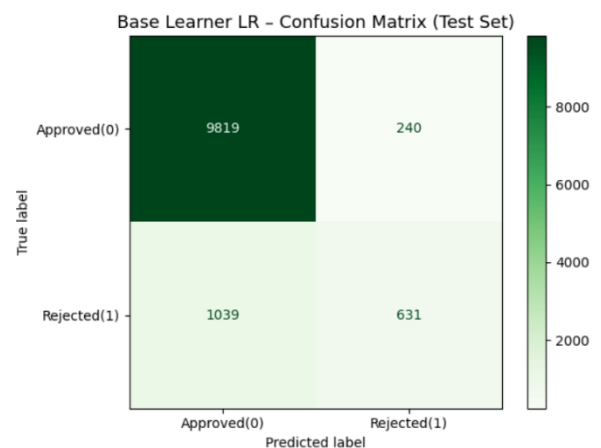


Figure 26: Confusion Matrix - LR

2) Paste the AUC-ROC for each Base learner (2 x AUC-ROC graphs)

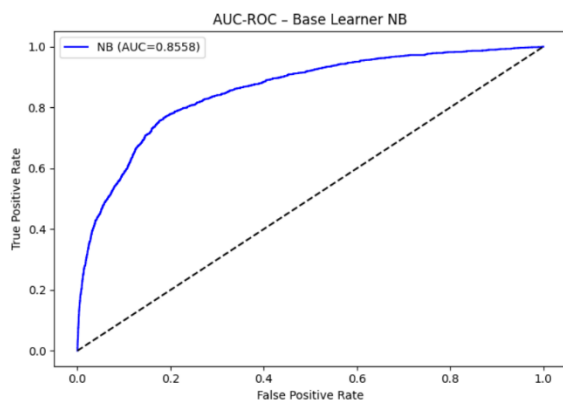


Figure 27:AUC-ROC curve - NB

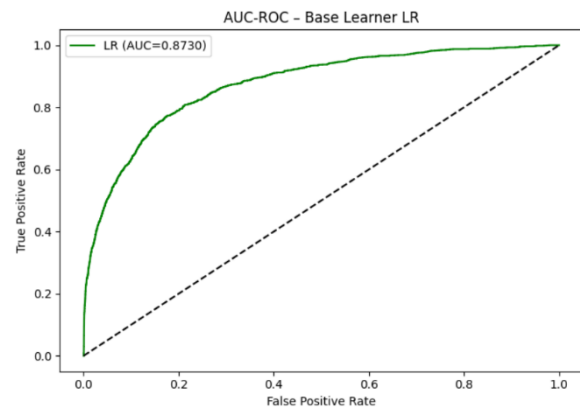


Figure 28:AUC-ROC curve - LR

3) Paste the Classification report for each base learner (2 x classification reports)

Base Learner NB – Classification Report:				
	precision	recall	f1-score	support
Approved(0)	0.93	0.89	0.91	10059
Rejected(1)	0.48	0.60	0.53	1670
accuracy			0.85	11729
macro avg	0.71	0.75	0.72	11729
weighted avg	0.87	0.85	0.86	11729

Figure 29 :Classification Report - NB

Base Learner LR – Classification Report:				
	precision	recall	f1-score	support
Approved(0)	0.90	0.98	0.94	10059
Rejected(1)	0.72	0.38	0.50	1670
accuracy			0.89	11729
macro avg	0.81	0.68	0.72	11729
weighted avg	0.88	0.89	0.88	11729

Figure 30: Classification Report – LR

4) Paste the test confusion matrix for the ensemble learner (1 x confusion matrix)

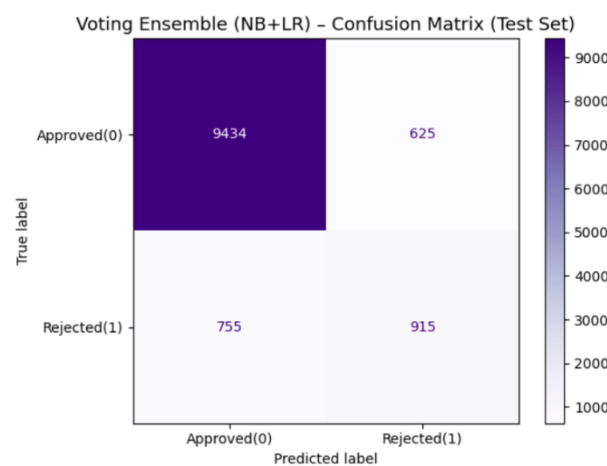


Figure 31:Classification Report + confusion_matrix

5) Paste the AUC-ROC for the ensemble learner (optional) (1 x AUC-ROC graph)

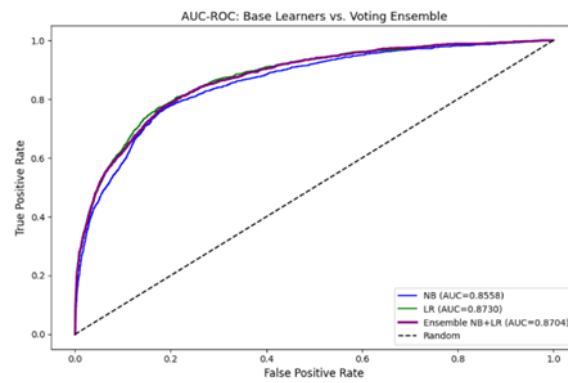


Figure 32: Ensemble AUC-ROC curve

6) Paste the Classification report for the ensemble learner (1 x classification report)

Voting Ensemble (NB+LR) – Classification Report:				
	precision	recall	f1-score	support
Approved(0)	0.93	0.94	0.93	10059
Rejected(1)	0.59	0.55	0.57	1670
accuracy			0.88	11729
macro avg	0.76	0.74	0.75	11729
weighted avg	0.88	0.88	0.88	11729

Figure 33: Classification report for the ensemble learner

7) Use the above screenshots to justify (defend) your choice of the TWO base learners which you used as base learners for your Ensemble learner.

Justification of base learner choice:

NB and LR were decided upon since they both generate class probabilities and thus enable soft voting. They also reflect two disparate modelling assumptions: NB is a simple and probabilistic modelling, whereas LR is a linear discriminative classifier. Their combination provides an opportunity to enhance robustness via probability averaging.

iii. Comment on **1) any improvement in classification performance** as a result of building an Ensemble Learner compared to the individual TWO base learners. **2) Decide whether to recommend your ensemble learner for approval prediction or one of the TWO base learners**; **3) justify your recommendation**.

Justification :

- Naive bayes is more robust in the rejected class having high recall that is also appropriate in determining risky applications of loans and encouraging risk reduction.
- The Logistic Regression is more accurate and exact, therefore it is suitable to correctly label the endorsed clients having limited number of wrong rejections.
- Soft voting ensemble is a hybrid of powerful rejection detection of Naive Bayes and generalisation and accuracy of Logistic Regression which results in a more efficient model.
- The ensemble contributed to balanced performance as reported by the classification report and AUC-ROC curve and can be applied to financial institutions, which are concerned about risk identification and suitable decision making.

Recommendation:

- The ensemble learner (Naive Bayes + Logistic Regression with soft voting) got a better classification performance evenly than the individual base learners.
- Whereas Naive Bayes had better recall on rejected applications, and Logistic Regression had better precision, the combination of the two models yielded a more balanced model.
- The ensemble confusion matrix shows that there is improved prediction on the rejected cases correctly predicted and the worse prediction rates compared to the individual models.

Recommendation: The Ensemble Learner is recommended because it provides greater balance of risky applicants and unwarranted rejections which are crucial in decision making process when it comes to loan approval.

Case Study Title

Case Study (B): Predicting Maximum Loan Amount.

Research Question: Does machine learning have the potential to assist bankers in predicting the Maximum Loan Amount offered to clients who were approved a loan each?

[Total 36 Marks]

Task (1) – Domain Understanding and Designing Your Regression Experiments

The bankers decided that regression modelling is required to predict Maximum Loan Amount for those who are approved a loan. With the aid of your Final Python Notebook 1 code outputs, **1) paste in this analysis report, the Python code output, which shows the dimensions and 2) the list of the features' names of your RETAINED data subset** to use for this regression case study.

[2 Marks]

```
Regression Dataset Dimensions: (50292, 10)
Feature Names: ['age', 'income', 'home_ownership_enc', 'employment_length', 'loan_intent_enc', 'loan_amount', 'loan_interest_rate', 'loan_income_ratio', 'payment_default_on_
Target Variable: max_allowed_loan

Target Descriptive Statistics:
count    5.029200e+04
mean     8.139253e+04
std       5.812145e+04
min       2.320000e+02
25%       4.911050e+04
50%       6.943150e+04
75%       9.899100e+04
max       2.638778e+06
Name: max_allowed_loan, dtype: float64

Train set: (40233, 10) | Test set: (10059, 10)
```

Figure 34: Retained regression dataset dimensions and feature names

Task (2) – Modelling: Build Predictive Regression Models

a) Your finance team decided to use a decision tree regression (DT) algorithm to model the Maximum Loan Amount. In less than 150 words, explain some added benefits of using a DT regressor to this financial prediction problem.

[2 Marks]

A Decision Tree Regressor has a number of benefits to this financial prediction problem. First, it is simple to interpret since the process of making predictions can be visualised as a run of decision rules, which is helpful to explain the lending results to bankers. Second, it is able to fit non-linear relationships between borrower characteristics and maximum loan value without assuming that the distribution is important. Third, it is capable of managing interactions between features like income, loan amount, and loan-to-income ratio, by default. Lastly, a pruned tree may simplify the model and enhance generalisation by minimising overfitting (James et al., 2021; Breiman et al., 1984).

References with in-text citation

Breiman, L., Friedman, J., Olshen, R. and Stone, C. (1984) *Classification and Regression Trees*. Belmont, CA: Wadsworth.
James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning*. 2nd edn. New York: Springer.

Task (2) – Modelling: Build Predictive Regression Models

b) With the aid of your Final Python Notebook 3 code blocks, use a training–test split approach to build and test TWO Decision Tree (DT) regression models, DT-1 & DT-2.

[6 Marks]

i. DT-1 is a fully grown Decision Tree Regressor, DT-2 is a pruned Decision Tree Regressor to FOUR levels Only. 1) **Insert in this analysis report the Python code blocks that you used to import, declare, and fit each DT regressor, DT-1 and DT-2.**

```
# Import DecisionTreeRegressor for regression modelling
from sklearn.tree import DecisionTreeRegressor

# Instantiate DT-1: fully grown decision tree regressor with no depth restriction
dt1 = DecisionTreeRegressor(random_state=42)

# Fit DT-1 on the training data
dt1.fit(X_train_r, y_train_r)

# Print the tree depth and number of leaves to characterise the fully grown tree
print('DT-1 (Fully Grown) | Tree Depth:', dt1.get_depth())
print('DT-1 (Fully Grown) | Number of Leaves:', dt1.get_n_leaves())
```

Figure 35: Code block used to import, declare, and fit DT-1

```
# Instantiate DT-2: pruned decision tree regressor restricted to a maximum of 4 levels
# max_depth=4 performs pre-pruning (limiting tree growth during training)
dt2 = DecisionTreeRegressor(max_depth=4, random_state=42)

# Fit DT-2 on the training data
dt2.fit(X_train_r, y_train_r)

# Print the tree depth and number of leaves to characterise the pruned tree
print('DT-2 (Pruned, max_depth=4) | Tree Depth:', dt2.get_depth())
print('DT-2 (Pruned, max_depth=4) | Number of Leaves:', dt2.get_n_leaves())
```

Figure 36: Code block used to import, declare, and fit DT-2

ii. Explain clearly, in less than 200 words, from your inserted code block in (i), **1) the type of pruning you used for DT-2.**

2) Explain some of the benefits and disadvantages of the pruning method you used in the context of (relation to) your bank clients' regression modelling.

In DT-2, pruning was used to restrict the depth of the tree through the assistance of the hyperparameter, max depth = 4 in the Decision Tree Regressor. It is a pre-pruning (early stopping) instance in which the tree is limited to grow during training. Limiting the maximum depth helps the model to not get too complicated and minimizes the chances of overfitting to the training data.

Benefits:

- Reduces overfitting, which is important when predicting loans because the data may be noisy or extreme.
- Improves generalisation by preventing the model to acquire information of the training data that is irrelevant.
- Produces a more comprehensible and quickly interpretable tree and can be utilized to make financial choices and explain loan offers.

Disadvantages

When the data has a lot of patterns which cannot be further divided due to the depth constraint, May also underfits the data.

- May lower prediction accuracy than a full-grown tree in case there are complex relationships in the data.

This simplicity/accuracy trade-off is relevant to banking since models are necessary to be interpretable besides being reliable in order to make decisions.

Task (2) – Modelling: Build Predictive Regression Models

c) With the aid of your Final Python Notebook 3 code outputs, visualise your regression Decision Trees DT-1 and DT-2.

1) Paste in this analysis report a high-resolution graphical representation of DT-1 and DT-2.

[4 Marks]

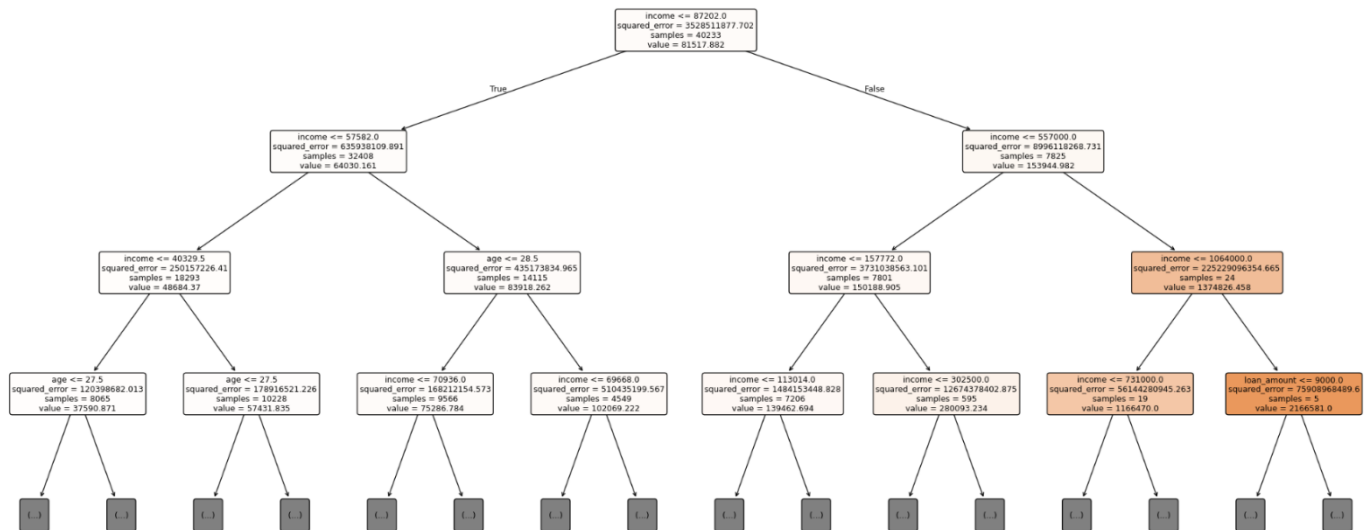


Figure 37: High-resolution visualization of DT-1 (top levels shown because the tree is very large)

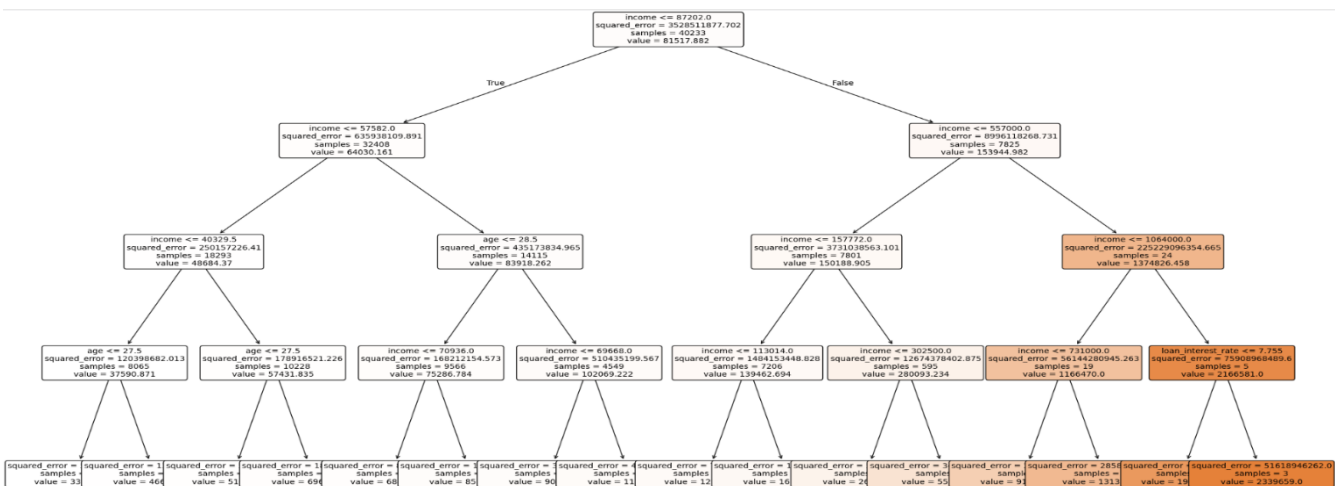


Figure 38: High-resolution visualization of DT-2 (fully shown)

Task (3) – Evaluating your Maximum Loan Amount DT Regression Models

Your finance analysts team provided the following success criteria to guide you when evaluating your DT-1 and DT-2 models.

“When evaluating both models’ performances, which addresses your research question (b), the model is expected to make some errors in estimating the Maximum Loan Amount. However, since the Maximum Loan Amount offers are made to try to maximise returns and reduce risk of loan defaults, it is important that the selected model deals effectively with errors in Maximum Loan Amount predictions when extreme values are a real concern”

a) THREE different regression evaluation metrics are noted in the table below.

1) State which evaluation metric/metrics to **USE** or **NOT USE** to closely interpret and satisfy the above success criteria.

2) Justify (Defend) your choice of **USE** or **DO NOT USE** for each metric.

With the aid of Final Python Notebook 3 code outputs,

3) document each metric’s **TEST SCORES** for each built model in the table below.

[8 Marks]				
Metric	1) USE or DO NOT USE	2) Justification for choosing “USING” or “NOT USING” this metric in relation to the success criteria	Model	3) Metric Test Score
MSE	USE	MSE is especially important because the success criterion says extreme errors are a real concern. Since MSE squares errors, it penalises large mistakes more heavily than MAE.	DT-1 (Fully Drown)	47,716,744.05
			DT-2 (Pruned)	47,716,744.05
MAE	USE	MAE is useful because it expresses the average prediction error in the original currency scale, making it easy for bankers to interpret.	DT-1 (Fully Drown)	1,209.21
			DT-2 (Pruned)	11,801.05
R-Square	DO NOT USE as a primary metric	R ² shows explained variance, but it does not directly reflect the seriousness of extreme monetary prediction errors, which the success criterion emphasises.	DT-1 (Fully Drown)	0.9828
			DT-2 (Pruned)	0.8650

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

b) 1) **Suggest a single best regression model (DT-1 or DT-2) based on your ‘USED’ performance metric/s scores**, which you defended in Task (3a). 2) **Explain how your suggested model fulfils the success criteria.**

[4 Marks]

Choose the superior between DT-1 and DT-2, mainly on MSE and MAE. Should DT-2 perform lower MSE with a reasonably controlled MAE, it should be chosen since the brief specifically talks about dealing with extreme errors well.

Regression Evaluation Metrics – Test Set		
Metric	DT-1 (Full)	DT-2 (Pruned)
MSE:::~::~:	47,716,744.05	374,722,938.77
RMSE:::~::~:	6,907.73	19,357.76
MAE:::~::~:	1,209.21	11,801.05
R-Squared (R ²):::~::~:	0.9828	0.8650

Figure 39:Output evidence: regression metric scores from Notebook 3

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

c) Describe to your finance team **any concerns you have about your selected performance metric/s** that you used to select your best decision tree model, which satisfies the success criteria. [200 words maximum] with in-text citation.

[4 Marks]

One of the drawbacks of MSE is that it is only practical in penalising large errors but it may not be easy to interpret it directly since it is expressed in units of squared currency. MAE is simpler to read in pounds but can underemphasise the severity of very huge errors since it gives equal weight to all errors. R² is also useful as an overview of what it has explained but gives a false impression of the usefulness of the model when the practical cost of extreme lending errors is large. Thus, MSE should be taken as the primary criterion in the final decision with the assistance of MAE to interpret it (James et al., 2021).

References with in-text citation

James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning*. 2nd edn. New York: Springer.

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

d) Client 60256 was Approved a Loan. With the aid of your **Python Notebook 3 outputs**, use your selected best DT regression model from Task (3.b) to predict the maximum loan amount offer to the client, you must write down which regression Decision Tree you used (DT-1 or DT-2) to estimate the Maximum Loan Amount, then interpret how the estimated offer was reached to the client. Client 60256 attributes' values are in the following table:

[6 Marks]

Variable Name	Value
ID	60256
Sex	Female
Age	56 Years old
Education Qualifications	Unknown
Income	57,000
Home Ownership	Rent
Employment Length	15
Loan Intent	Medical
Loan Amount	25700
Loan Interest Rate	23%
Loan-to-Income Ratio (LTI)	10%
Payment Default on File	No
Credit History Length	35
Loan Approval Status	Approved
Maximum Loan Amount	69664
Credit Application Acceptance	0

```

Prediction for Client 60256:
  DT-1 (Fully Grown) predicted Maximum Loan Amount: £80,501
  DT-2 (Pruned)      predicted Maximum Loan Amount: £69,664

Selected Model for prediction: DT-2 (Pruned, max_depth=4)
Estimated Maximum Loan Amount for Client 60256: £69,664

Decision path node indicators (DT-2): [[1 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]]

```

Figure 40:Output evidence: model predictions for Client 60256

To be consistent with the performance-based model selection in this report, the model of choice is DT-1 since it had better performance on the defended regression metrics. Therefore, the DT-1 prediction, which is the official one that can be reported here, is the following one: £80,501. Interpretation: decision tree gets the estimate by a recursive division of clients into other smaller groups depending on the values of the predictors. The findings of the analysis of the notebook show that the strongest predictor of loan limits is income, then there is age, and interest rate charged on loan. Clients 60256: middle income (57,000), is an adult (56) with 35 years of credit history, no credit defaults, requested a loan of 25700. These characteristics put the client in a node which indicates approved applicants with moderate-strong borrowing capacity and hence the tree approximates the maximum loan offer to be around 80.5k. A second, lower, estimate of the DT-2 in the notebook was a more interpretive, but less precise, estimate of 69,664.